

Reproduce-then-Extend — Wine Quality (Food Science / Chemometrics) (Day 1)

Intro to Machine Learning · Bruce A. Desmarais

July 03, 2026

Published study. Cortez, P. et al. (2009). “Modeling wine preferences by data mining from physicochemical properties.” *Decision Support Systems* 47(4):547–553. The baseline is a **multiple (linear) regression** of sensory quality on 11 physicochemical measurements. Data: UCI (Vinho Verde red wine, n = 1599). This example illustrates **polynomial regression** — and using cross-validation to judge whether it is worth it.

1 Background

Cortez and colleagues ask whether **objective physicochemical measurements** of a wine can predict its **subjective sensory quality** (the median score of blind tasters), which would support quality certification and production decisions. They model quality from 11 physicochemical properties using several methods, with **multiple linear regression** as the interpretable baseline. The **primary conclusion** is that physicochemical properties do carry real signal – **alcohol** (positive) and **volatile acidity** (negative) matter most – but the relationship is modest and nonlinear, so flexible models (support vector machines in their paper) predict quality better than the linear regression.

2 Data and codebook

Unit of analysis: an individual red ‘Vinho Verde’ **wine sample**; n = 1,599.

Variable	Definition
quality	sensory quality, median of ≥ 3 blind tasters (0-10) (outcome)
fixed.acidity	fixed (tartaric) acidity (g/L)
volatile.acidity	volatile (acetic) acidity (g/L)
citric.acid	citric acid (g/L)
residual.sugar	residual sugar (g/L)
chlorides	chlorides / salt (g/L)
free.sulfur.dioxide	free SO ₂ (mg/L)
total.sulfur.dioxide	total SO ₂ (mg/L)
density	density (g/mL)
pH	pH
sulphates	potassium sulphate additive (g/L)
alcohol	alcohol (% by volume)

3 Descriptive results

```
url <- "data/winequality-red.csv"
d <- read.csv(url, sep = ";")
names(d) <- make.names(names(d))
preds <- setdiff(names(d), "quality")
```

```
round(t(sapply(d, function(x) c(mean = mean(x), sd = sd(x), min = min(x), max =
  ↪ max(x))))), 3)
```

```
##              mean      sd   min    max
## fixed.acidity    8.320  1.741 4.600 15.900
## volatile.acidity 0.528  0.179 0.120  1.580
## citric.acid      0.271  0.195 0.000  1.000
## residual.sugar   2.539  1.410 0.900 15.500
## chlorides        0.087  0.047 0.012  0.611
## free.sulfur.dioxide 15.875 10.460 1.000 72.000
## total.sulfur.dioxide 46.468 32.895 6.000 289.000
## density          0.997  0.002 0.990  1.004
## pH               3.311  0.154 2.740  4.010
## sulphates        0.658  0.170 0.330  2.000
## alcohol          10.423  1.066 8.400 14.900
## quality          5.636  0.808 3.000  8.000
```

```
par(mfrow = c(1, 2))
par(mar = c(4, 4, 2.5, 1))
barplot(table(d$quality), col = "#a6cee3", border = "white", main = "Outcome: quality",
  xlab = "sensory score", ylab = "count")
cors <- sort(sapply(d[preds], function(x) cor(x, d$quality)))
par(mar = c(4, 8, 2.5, 1))
barplot(cors, horiz = TRUE, las = 1, cex.names = 0.8,
  col = ifelse(cors > 0, "#1f78b4", "#e31a1c"), main = "Correlation with quality",
  xlab = "correlation")
abline(v = 0, col = "grey50")
```

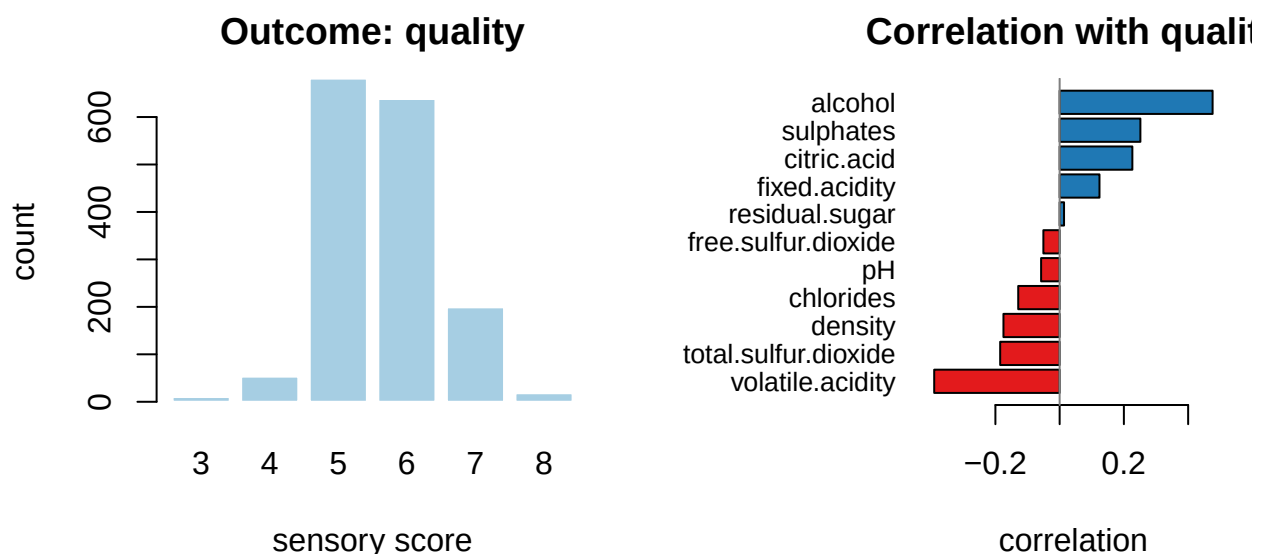


Figure 1: Quality-score distribution (left) and each predictor's correlation with quality (right).

4 Reproduce the published regression

```
mr <- lm(quality ~ ., data = d)
summary(mr)

##
## Call:
## lm(formula = quality ~ ., data = d)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -2.68911 -0.36652 -0.04699  0.45202  2.02498
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    2.197e+01  2.119e+01   1.036   0.3002
## fixed.acidity    2.499e-02  2.595e-02   0.963   0.3357
## volatile.acidity -1.084e+00  1.211e-01  -8.948 < 2e-16 ***
## citric.acid     -1.826e-01  1.472e-01  -1.240   0.2150
## residual.sugar   1.633e-02  1.500e-02   1.089   0.2765
## chlorides       -1.874e+00  4.193e-01  -4.470 8.37e-06 ***
## free.sulfur.dioxide  4.361e-03  2.171e-03   2.009   0.0447 *
## total.sulfur.dioxide -3.265e-03  7.287e-04  -4.480 8.00e-06 ***
## density         -1.788e+01  2.163e+01  -0.827   0.4086
## pH              -4.137e-01  1.916e-01  -2.159   0.0310 *
## sulphates        9.163e-01  1.143e-01   8.014 2.13e-15 ***
## alcohol         2.762e-01  2.648e-02  10.429 < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.648 on 1587 degrees of freedom
## Multiple R-squared:  0.3606, Adjusted R-squared:  0.3561
## F-statistic: 81.35 on 11 and 1587 DF, p-value: < 2.2e-16
```

Confirmation against the published study. The multiple regression attains in-sample $R^2 = 0.361$, matching the modest linear fit Cortez et al. report. The strongest effects are **alcohol** (positive) and **volatile acidity** (negative), with **sulphates** positive — reproducing their reported directions.

5 Polynomial regression extension

Sensory quality may not respond to **alcohol** at a constant rate. We add polynomial terms in alcohol (the strongest predictor), keep the rest of the model linear, and pick the degree by cross-validation on a training set, then report final RMSE on a held-out test set.

```
set.seed(2026)
others <- setdiff(preds, "alcohol")
te <- sample(nrow(d), round(0.30 * nrow(d)))           # hold out a 30% test set up
  ↪ front
tr <- setdiff(seq_len(nrow(d)), te)

# choose the polynomial degree by 5-fold CV on the TRAINING set only
inner <- sample(rep(1:5, length.out = length(tr)))
cv_rmse <- sapply(1:4, function(deg) {
  r <- numeric(5)
  for (k in 1:5) {
```

```

itr <- tr[inner != k]; ite <- tr[inner == k]
form <- reformulate(c(sprintf("poly(alcohol, %d, raw = TRUE)", deg), others),
  ↪ "quality")
r[k] <- sqrt(mean((d$quality[ite] - predict(lm(form, data = d[itr, ]), d[ite, ]))^2))
}
mean(r)
})
names(cv_rmse) <- paste0("degree ", 1:4)
best_deg <- which.min(cv_rmse)
round(cv_rmse, 4)
# CV RMSE per degree (training set)

```

```

## degree 1 degree 2 degree 3 degree 4
## 0.6414 0.6423 0.6426 0.6427

```

```

# refit at the CV-chosen degree on the full training set; score ONCE on the untouched
  ↪ test set
form_best <- reformulate(c(sprintf("poly(alcohol, %d, raw = TRUE)", best_deg), others),
  ↪ "quality")
test_poly <- sqrt(mean((d$quality[te] - predict(lm(form_best, data = d[tr, ]), d[te,
  ↪ ]))^2))
test_lin <- sqrt(mean((d$quality[te] - predict(lm(reformulate(c("alcohol", others),
  ↪ "quality"), data = d[tr, ]), d[te, ]))^2))
cat(sprintf("CV-selected degree: %d. Held-out test RMSE: polynomial = %.4f vs linear =
  ↪ %.4f",
  best_deg, test_poly, test_lin))

```

```

## CV-selected degree: 1. Held-out test RMSE: polynomial = 0.6710 vs linear = 0.6710

```

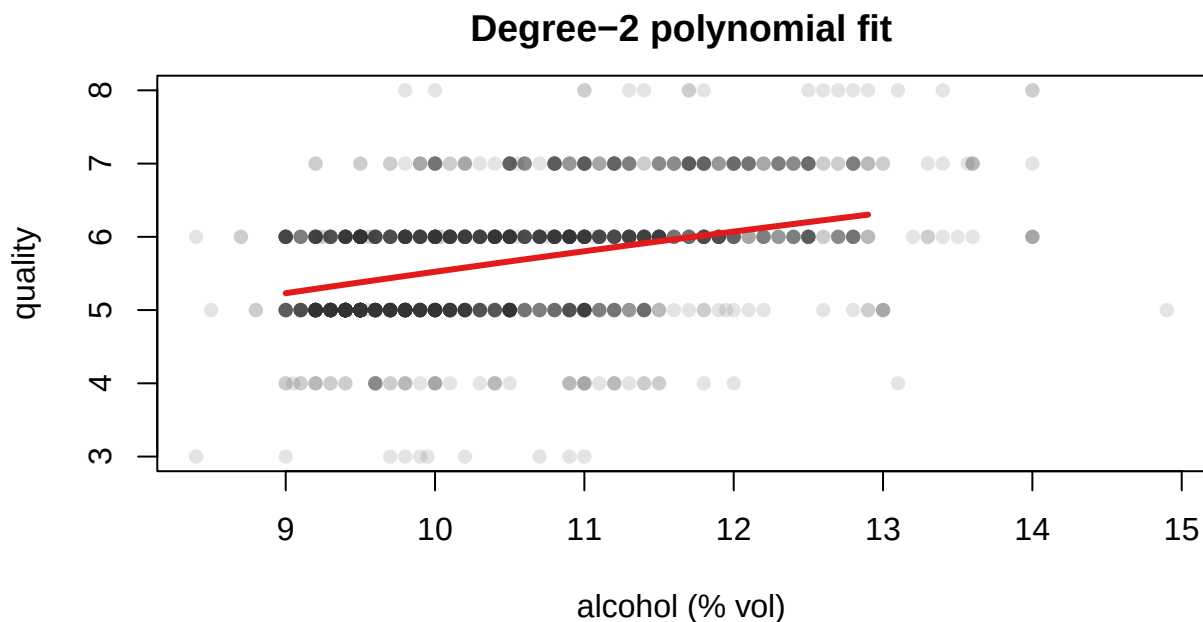


Figure 2: Degree-2 polynomial fit of quality on alcohol, holding the other predictors at their means.

The fitted curve bends gently, but the **held-out test error barely moves** — a single polynomial term in alcohol adds almost nothing here. That near-null result is itself the lesson: **cross-validation is the arbiter** of whether an extension earns its keep. For wine, the useful curvature is *multivariate* (interactions among

alcohol, acidity and sulphates), which a one-variable polynomial cannot reach.

6 Takeaway

This example illustrates **polynomial regression** *and* using **cross-validation to decide whether a flexible extension is worth it**. Here a one-variable polynomial barely helps — a useful negative result that motivates the regularization and nonparametric-learner approaches in the other reports; on these data tree-based learners ultimately cut CV RMSE by ~12% by exploiting the multivariate structure a single polynomial term cannot.

7 Recommended exercises

1. Fit a polynomial in a predictor other than alcohol (e.g., volatile acidity) and check cross-validated RMSE.
2. Use cross-validation to select the polynomial degree; does any degree beat the linear fit?
3. Compare the polynomial regression to a random forest on the same folds.